Smart Vending Machine for Digital Goods

Project Overview

Scenario: In this digital era, imagine a Smart Vending Machine where you can purchase digital items such as e-books, software licenses, and media files. Designed for users on the go, this vending machine offers instant purchases and secure payment options, enabling access to a range of digital products anytime. With a user-friendly interface, you can browse items, add them to a cart, manage quantities, and complete payments, with stock automatically updating on successful purchases.

Project Goals: The project models a client-server vending machine system, where the client handles the user interface, item selection, and payment, while the server manages product inventory, order processing, and transaction records.

1. **Item Viewing and Selection:** View available digital items (e.g., e-books, software) with details on prices and availability.
2. **Cart Management:** Add items to a virtual cart, manage quantities, and view a detailed cart summary.
3. **Payment Simulation:** Choose from payment options (e.g., credit, debit) and receive confirmation upon purchase.
4. **Order History:** Store previous transactions and display order details on request. Automatic Stock Update: The server updates the inventory after each purchase.

Submission Requirements:

The submission for the project must include both the **completed code** and a **1000-word report**. The deadline for code submission is provided in the submission link. Submit a single `.zip` file containing a folder with the following:

1. **Python Code**:
   - Include all Python files necessary for the project to function, such as `client.py`, `server.py`, and any additional modules used in the application.
2. **Data Files**:
   - Add any required data files, such as `.txt`, `.csv`, or `.db` files, to support product inventory and transaction records. If an SQLite database is used, include the `.db` file.
3. **Images**:
   - If the project includes a GUI that uses images, add these images to an `images` folder within the main submission directory.
4. **Report**:

- Submit a PDF report titled `report.pdf` containing the following sections:
  - **Credentials**: Include your name, surname, and student number.
  - **UML Diagram**: Include a UML diagram or other visual representation of the project's structure, depicting core components such as `Product`, `Cart`, and `Order`, and illustrating client-server interactions.
  - **Implementation Plan**: Provide an outline of your approach to developing the vending machine, including the initial text-based setup, the move to OOP structure, and the implementation of client-server communication. Describe the steps taken to manage inventory and process transactions, and discuss how data is stored and accessed.
  - **Running Instructions**: Include clear instructions on how to run the application from the terminal or command line, including any required installations or dependencies.
  - **Reflection**: Offer reflections on your development experience. Topics to include are:
    - How each stage of the project (inventory setup, OOP refactoring, client-server structure, database integration, GUI) contributed to your learning and understanding of Python programming.
    - Challenges faced in developing the client-server model, such as data handling, error management, or socket communication, and your solutions.
    - Insights gained from implementing real-world functionalities, such as dynamic inventory and transaction processing.
    - Potential improvements or additional features you would consider adding, given more time or advanced skills.

Technical Requirements:

- The code must be developed using **Python 3**.
- Ensure the application runs directly from the terminal or command line (CMD) on a standard Python environment.
- the application should run on a local setup and that any external libraries used should be specified in the `Running Instructions`.

Organize the `.zip` file logically to facilitate easy access to code, data, images, and the report for evaluation.

Instructions by Week

Create a **Digital Vending Machine** application where users can view digital products, add them to a cart, and complete purchases. Start with a text-based structure, and over the coming weeks, add features like Object-Oriented Programming, client-server communication, a database, and a graphical interface.

Weeks 6-7: Text-Based Application with Core Functions

1. **Set Up Product Inventory**:
   - Create a text file called `inventory.txt` to store information about each product. Format each line in the file with four pieces of data: `ID`, `Product Name`, `Price`, and `Stock`. Separate each item by commas. Example:
     ```plaintext
     1, Python Programming eBook, 10.99, 25
     2, Photo Editing License, 29.99, 15
     3, Study Playlist MP3, 5.99, 50
     ```
2. **Write `load_inventory()` Function**:
   - Write a function called `load_inventory()` that reads `inventory.txt` and loads the products into a list of dictionaries. Each dictionary should represent one product with keys like `id`, `name`, `price`, and `stock`.
   - Test by printing the loaded products to confirm the function works correctly.
3. **Write `view_products()` Function**:
   - Write a function called `view_products()` to display all products in the inventory. Include the ID, name, price, and stock for each item.
4. **Set Up Cart and Write `add_to_cart()` Function**:
   - Create an empty dictionary called `cart` to store selected items.
   - Write an `add_to_cart(product_id, quantity, inventory)` function that:
     - Takes the product ID and quantity as inputs.
     - Checks if enough stock is available.
     - If available, adds the product and quantity to the `cart` and reduces the stock in `inventory`.
5. **Write `view_cart()` Function**:
   - Write a function called `view_cart()` that displays the current contents of the cart. This function should list each product in the cart, the quantity, and the total price.
6. **Write `checkout()` Function**:
   - Write a `checkout()` function that:
     - Displays a confirmation summary of the cart's contents.
     - Empties the cart after checkout.
     - Updates the inventory file by calling `save_inventory()`.
7. **Write `save_inventory()` and `save_transaction()` Functions**:
   - Write `save_inventory(inventory)` to rewrite `inventory.txt` with the updated stock after each purchase.
   - Write `save_transaction(cart)` to log each purchase in a `transactions.txt` file for future reference.

(Note: function names are recommended for clarity but can be modified as needed)

Week 7: Refactor with Object-Oriented Programming

1. **Create `Product` Class**:
   - Define a `Product` class with `id`, `name`, `price`, and `stock` attributes.
   - Add an `update_stock(quantity)` method to handle stock adjustments when items are added to the cart.
2. **Create `Cart` Class**:
   - Define a `Cart` class with an `items` dictionary attribute to store products and quantities.
   - Add methods:
     - `add_item(product, quantity)` to add products to the cart.
     - `view_items()` to display cart contents and total cost.
     - `clear_cart()` to empty the cart after checkout.
3. **Create `VendingMachine` Class**:
   - Define a `VendingMachine` class to manage inventory and checkout.
   - Add methods to load the inventory, display products, process checkout, and save transactions.

Refactor the existing functions from Weeks 6-7 into these classes.

Weeks 8-9: Convert to Client-Server Model with Socket Communication

1. **Set Up the Server Program**:
   - Write a server program to handle inventory and transaction processing. Load the inventory from `inventory.txt` on startup.
   - Implement the following functions:
     - **`load_inventory()`**: Reads product data from `inventory.txt` into a dictionary.
     - **`get_product_list()`**: Returns the list of all products with IDs, names, prices, and stock levels to the client.
     - **`process_checkout(cart)`**: Verifies stock, updates inventory, and confirms purchases based on cart data from the client.
     - **`save_transaction(cart)`**: Records completed transactions in `transactions.txt`.
2. **Set Up the Client Program**:
   - Write a client program that connects to the server.
   - Implement the following client-side functions:
     - **`display_products()`**: Requests product data from the server and displays it to the user.
     - **`add_to_cart(product_id, quantity)`**: Adds items to the local cart and sends the details to the server for inventory verification.

    - `view_cart()`: Displays the cart contents and the total cost.
    - `request_checkout()`: Sends cart data to the server for processing and waits for confirmation.
3. **Implement Client-Server Communication**:
    - Use `socket` programming for communication:
        - **Client**: Sends requests (e.g., view products, add to cart, checkout).
        - **Server**: Receives requests, processes them, and sends responses.

Week 9: Implement SQLite Database on the Server

1. **Set Up SQLite Database**:
    - Create an SQLite database for inventory and transactions.
    - Set up tables:
        - `products`: Stores product details like ID, name, price, and stock.
        - `transactions`: Records each completed transaction.
2. **Modify Server Functions to Use Database**:
    - **Database Functions**:
        - `load_inventory_from_db()`: Retrieves product data from the database on server startup.
        - `update_inventory_in_db(product_id, quantity)`: Updates stock in the `products` table after a purchase.
        - `save_transaction_in_db(cart)`: Inserts completed transactions into the `transactions` table.
    - Update the server's `get_product_list()` and `process_checkout()` functions to interact with the database instead of the text file.
3. **Test Database Integration**:
    - Test each function to ensure that inventory and transaction records are correctly saved and retrieved from the database.

Week 10: Add GUI with Tkinter for Client

1. **Create Main Product Display Window**:
    - Use Tkinter to build a window that shows all products.
    - Add buttons next to each product for adding items to the cart.
2. **Add Cart Management Window**:
    - Create a cart summary window with options to adjust item quantities, view the total cost, and proceed to checkout.
3. **Add Checkout Window**:
    - Add a final confirmation window displaying the total cost and options to confirm or cancel the transaction.

4. **Connect GUI Buttons to Functions**:
    - Set up buttons to trigger client-side functions:
        - **Add to Cart**: Calls `add_to_cart()` and updates the display.
        - **Checkout**: Calls `request_checkout()` to send cart data to the server for final processing.
5. **Test Full GUI Flow**:
    - Run the client program, testing each step from viewing products to adding items to the cart, and completing checkout with server interaction.

Summary of Client and Server Functions

Client-Side:

- **User Interface**: Display products, manage cart, and process checkout.
- **Cart Management**: Functions to add/view items, calculate totals, and confirm purchases.
- **Client-Server Communication**: Functions to send requests to the server (view products, checkout) and handle responses.

Server-Side:

- **Inventory Management**: Functions to load inventory, check stock, update quantities, and save transactions.
- **Database Management**: Functions to retrieve and update inventory and transaction data in SQLite.
- **Transaction Processing**: Functions to process the client's cart, confirm purchases, and record orders.

(Note: function/class/methods names are recommended for clarity but can be modified as needed)

Marking Criteria

Criteria	Details	Marks Range
Foundational Requirements		**0-45%**
Report Quality	- PDF basic report and clear structure of files are expected at minimum, including credentials, a design overview, and a UML or structural diagram illustrating main components like `Product`, `Cart`, and `Order`.	
Code Submission	- Python files for client and server included. - Data files (e.g., `.txt` or `.csv`) for inventory and transactions are present. - Code includes essential functions, covering: • **Inventory Management**: Loads and displays	

Criteria	Details	Marks Range
	products. • **Data Storage**: Reads and updates inventory and cart data.	
Basic Functionality	- Functional text-based interface, allowing users to view products, add items to cart, and complete checkout. - Basic client-server communication that sends inventory data and transaction requests between client and server.	
Intermediate Requirements		**46-70%**
Report Quality	- Report includes a structured implementation plan, refined UML diagram, and reflections on challenges and learning.	
Enhanced Code Functionality	- Code refactored with Object-Oriented Programming (OOP), using classes like `Product`, `Cart`, and `VendingMachine`. - Established client-server communication with socket programming to enable: • Cart management and inventory updates on server. • Transaction processing between client and server.	
File/Data Management	- Effective use of data files or initial SQLite database integration to handle inventory and transaction records.	
Advanced Requirements		**70-90%**
Detailed Report	- Detailed implementation plan explaining each development phase, full UML diagrams, and design decisions. - Insightful reflections, including on challenges of implementing OOP and client-server interactions.	
Fully Developed Code	- Complete integration with SQLite database for inventory and transaction data. - Client-server model fully implemented, handling data requests and cart updates efficiently. - Tkinter GUI developed with intuitive product browsing, cart, and checkout functionality.	
Documentation	- Code is well-documented, with clear comments explaining all key sections and functions for ease of understanding.	
Exceptional Work (Extra 10%)		**+10%**
Dynamic Inventory and Analytics	- Implements Matplotlib for graphical analysis of purchase trends or stock distribution on the client side.	
Advanced Client-Server Features	- Real-time or simulated stock updates on the server, with automatic reflection on the client side. - Support for handling multiple clients interacting with separate sessions.	

In [ ]: